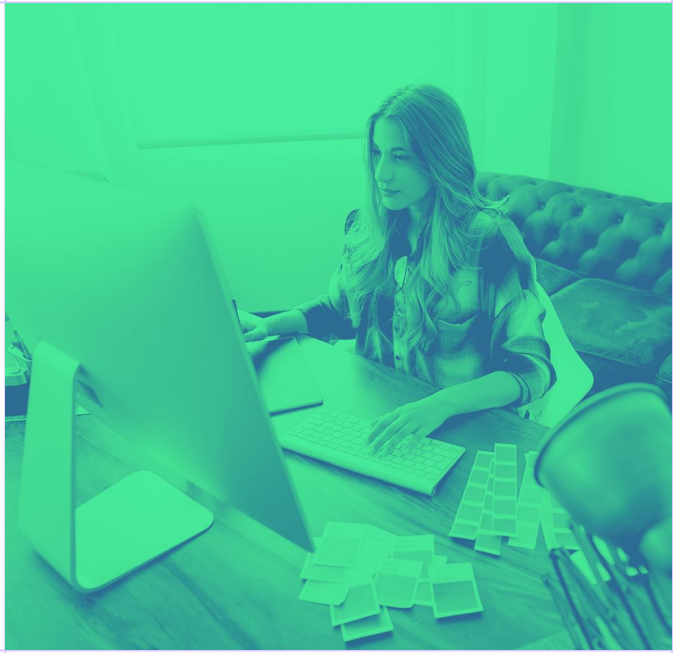
Mise en ligne

AWS - Gitlab - CICD

STIENNE Dylan



Next

Objectifs du module

- Comprendre les bases du cloud et de AWS
- Connaître les services élémentaires d'AWS
- Savoir mettre en place une CICD de A à Z
- Mettre en place une infrastructure simple sur AWS
- Déployer des fronts

En gros on va refaire vercel

Avant AWS, pourquoi parler du cloud ?

- Le cloud est devenu la norme de l'IT moderne
- Utilisé par les startups comme par les États
- Présent dans presque toutes les applications du quotidien
- AWS est un acteur, pas le cloud lui-même

Qu’est-ce que le Cloud Computing ?

- Accès à des ressources informatiques via Internet
- Ressources mutualisées et à la demande
- Pas de gestion physique du matériel
- Paiement à l’usage

Next

On-premise vs Hébergement vs Cloud

- On-premise : serveurs physiques internes
- Hébergement classique : serveur loué mais figé
- Cloud : ressources dynamiques, élastiques
- Différence clé : l’élasticité

Next

Les modèles de service Cloud

- **IaaS** : Infrastructure (serveurs, réseau)
- **PaaS** : Plateforme (bases, runtimes)
- **SaaS** : Application finale (outil clé en main)

AWS fournit surtout IaaS et PaaS

AWS

Next

En quelques mots

- Amazon Web Services
- Fournisseur de cloud public
- Plus de 200 services
- Leader mondial du cloud

Pourquoi AWS a été créé ?

- Besoins internes d'Amazon
- Infrastructure scalable pour le e-commerce
- Mutualisation devenue un produit
- AWS lancé en 2006

Régions et zones de disponibilité

- Une région = zone géographique
- Une région contient plusieurs AZ
- Chaque AZ = data center indépendant
- Base de la haute disponibilité

Une plateforme globale

- Présent sur tous les continents
- Réseau privé mondial
- Services interconnectés
- Très forte résilience

Pourquoi utiliser AWS ?

Next

Avantages techniques d'AWS

- Scalabilité automatique
- Haute disponibilité native
- Résilience aux pannes
- Déploiement rapide

Pourquoi utiliser AWS ?

Avantages économiques

- Pas d'investissement initial (CAPEX)
- Paiement à l'usage (OPEX)
- Ajustement en temps réel
- Réduction du time-to-market

AWS pour les développeurs

- API partout
- Infrastructure as Code
- Automatisation possible
- Fort écosystème d'outils

AWS pour les entreprises

- Sécurité avancée
- Conformité (ISO, RGPD, etc.)
- Support entreprise
- Évolutivité mondiale

Qui utilise AWS ?

Next

Qui utilise AWS ?

Typologies d’utilisateurs AWS

- Startups
- PME / ETI
- Grands groupes
- Secteur public
- Recherche et universités

Next

Qui utilise AWS ?

Startups et AWS

- Démarrage rapide
- Peu de coûts initiaux
- Forte croissance possible
- Serverless très utilisé

Qui utilise AWS ?

Grandes entreprises sur AWS

- Netflix
- Airbnb
- Spotify
- Twitch
- Doctolib

Next

Qui utilise AWS ?

Pourquoi ces entreprises choisissent AWS ?

- Scalabilité massive
- Disponibilité mondiale
- Sécurité et conformité
- Innovation rapide (IA en ce moment)

AWS et ses concurrents

Next

Les grands acteurs du cloud

- AWS
- Microsoft Azure
- Google Cloud Platform
- OVHcloud
- Scaleway

AWS vs Azure vs GCP

- AWS : plus complet, plus complexe
- Azure : fort en entreprise Microsoft
- GCP : data, IA, Kubernetes

Choix stratégique selon le contexte

Cloud souverain et alternatives

- OVHcloud (Europe)
- Scaleway (France)
- Souveraineté des données
- Moins de services managés

Combien ça coûte AWS ?

Next

Le modèle de tarification AWS

- Pas d'abonnement global
- Facturation à l'usage
- Prix par service
- Très granulaire

Combien ça coûte AWS ?

Ce qui coûte sur AWS

- Temps de calcul
- Stockage
- Trafic sortant
- Services managés

Next

Les coûts cachés

- Logs (CloudWatch)
- Trafic inter-services
- Bases de données
- Mauvais dimensionnement

Message clé sur les coûts

- AWS n’est pas cher par défaut
- Une mauvaise architecture coûte cher
- Le design est essentiel
- Anticiper les coûts dès le début

Pourquoi ne pas utiliser AWS ?

Next

Pourquoi ne pas utiliser AWS ?

À votre avis ?

- AWS est leader du cloud
- Utilisé par les plus grandes entreprises
- Très riche fonctionnellement
- Pourtant, ce n’est pas toujours le bon choix

Pourquoi ne pas utiliser AWS ?

AWS n’est pas toujours simple

- Courbe d’apprentissage élevée
- Beaucoup de services (trop ?)
- Concepts parfois complexes (IAM, réseau)
- Mauvais choix possibles quand on débute

Pourquoi ne pas utiliser AWS ?

Le risque de sur-architecture

- Tendance à empiler trop de services
- Complexité inutile pour des besoins simples
- Difficile à maintenir
- Difficile à expliquer à une équipe junior

Pourquoi ne pas utiliser AWS ?

Le coût mal maîtrisé

- Facturation très granulaire
- Coûts invisibles au départ
- Trafic, logs, services managés
- Facture qui explose sans alerte

Next

Pourquoi ne pas utiliser AWS ?

Le vendor lock-in

- APIs propriétaires
- Services très intégrés
- Migration vers un autre cloud complexe
- Dépendance stratégique à long terme

Pourquoi ne pas utiliser AWS ?

Quand AWS n’est PAS le bon choix

- Petit projet personnel simple
- Budget très limité
- Équipe très junior sans encadrement
- Besoin fort de souveraineté locale

Pourquoi ne pas utiliser AWS ?

Ce que AWS n’est PAS

- AWS n’est pas “plug and play”
- AWS ne remplace pas l’architecture
- AWS ne corrige pas un mauvais design
- AWS ne protège pas des mauvaises décisions

Pourquoi ne pas utiliser AWS ?

Alors... pourquoi on l'utilise quand même ?

- Écosystème extrêmement riche
- Scalabilité réelle
- Services managés puissants
- Standard de fait du marché

Comment et pour combien développer Netflix ou Uber sans AWS ?

Présentation des services AWS

Next

Services de calcul

- **EC2** : machines virtuelles
- **Lambda** : fonctions serverless
- **Auto Scaling** : adaptation automatique
- **ECS / Fargate** : conteneurs

Services de stockage

- **S3** : stockage objet
- **EBS** : disque pour EC2
- Snapshots et backups
- Durabilité élevée

Bases de données

- **RDS** : bases relationnelles
- **DocumentDB** : NoSQL Mongo-like
- **ElasticSearch** : recherche et logs
- Services managés = simplicité

Réseau et distribution

- **VPC** : réseau privé
- **Load Balancer** : répartition du trafic
- **Route 53** : DNS
- **CloudFront** : CDN

Serverless & APIs

- Lambda
- API Gateway
- EventBridge
- Step Functions

Messaging et asynchrone

- **SNS** : publication / abonnement
- **SQS** : file de messages
- Découplage des services
- Scalabilité améliorée

Sécurité et identité

- **IAM** : permissions
- **Cognito** : authentification
- **Secrets Manager** : gestion des secrets
- Sécurité partagée

Monitoring et observabilité

- **CloudWatch**
- Logs
- Métriques
- Alertes

IA et services avancés

- **Rekognition**
- Analyse d'images et vidéos
- Services prêts à l'emploi
- Paiement à l'usage

Découverte de la Console AWS

Next

TP AWS

Next

Consignes

Contexte
Vous êtes une startup, vous voulez héberger une image (Ex. logo) et la rendre accessible publiquement sans serveur et à moindre coût.

Objectif
Héberger une image en ligne avec AWS

Contraites
- Utiliser uniquement la console AWS
- Utiliser le service s3

Résultat attendu
Une image accessible via une URL publique

Étape 1 - Créer un bucket S3

- Aller dans la console AWS
- Ouvrir le service S3
- Créer un bucket
- Choisir une région
- Nom unique et global

Étape 2 - Configurer l’accès public

- Désactiver le “Block Public Access”
- Comprendre pourquoi c’est bloqué par défaut
- Assumer la responsabilité de l’accès public

Étape 3 - Uploader une image

- Choisir une image locale
- L'uploader dans le bucket
- Vérifier qu'elle apparaît dans la liste des objets

Étape 4 - Rendre l'image accessible

- Modifier les permissions de l'objet
- Autoriser la lecture publique
- Comprendre la notion de policy / ACL

Étape 5 - Tester l'accès

- Copier l'URL de l'image
- L'ouvrir dans un navigateur
- Vérifier qu'elle est visible sans être connecté à AWS

Étape 6 - Créer une distribution CloudFront

- Utiliser le service CloudFront
- Obtenir une url CloudFront pour votre fichier
- **BONUS** Lier votre nom de domaine via le service Route 53

Suppression

- S3
- CloudFront
- Route53

Sinon vous allez payer à vie

Mise en place gitlab

Repository, arborescence, Docker (dev container), Script de deploiement des assets, CI/CD associée

Next

Rendu
Déployer un Nuxt

Next

Rendu - Déployer un Nuxt

Partie 1

- Initialiser un projet Nuxt dans /code de votre répo

- Déployer à la main votre projet sur AWS
  - Création d'un bucket s3
  - Création d'une distribution

- Faire un build du projet

- Mettre le code buildé sans le s3 à la main

- Trouver les quelques configurations restantes pour que ça marche

Venir me montrer que ça marche

Rendu - Déployer un Nuxt

Partie 2

- Créer un script deploy.nuxt.py
  - install
  - build (generate)
  - clear ancien s3
  - push sur le s3

- Attention au cache
  - changer un mot sur le front
  - redeployer avec le script
  - le mot doit tout de suite apparaître en ligne

- Actualiser la CICD pour y mettre ce script

Rendu - Déployer un Nuxt

Partie 3

Le but est d'ajouter une étape de CI au pipeline existant.
Cette étape devra être intégrée au processus de développement afin d'automatiser les vérifications de qualité du code.
Concrètement, elle consistera à exécuter des checks de linting lors de chaque exécution de la CI, permettant de s'assurer que le code respecte les conventions et standards définis pour le projet.

- Créer le script
- Lancer le script dans la CICD
- Le pipeline doit s'arrêter si un souci de lint est détecté

Rendu - Déployer un Nuxt

Partie 4

Il faut maintenant gérer plusieurs environnements.
Le projet devra désormais distinguer un environnement de développement (dev) et un environnement de production (prd), avec des configurations adaptées à chacun.

- Il faudra surement créer de nouvelles ressources AWS
- Il faudra surement adapter les scripts existants sans les dupliquer
- Les variables d'env ne doivent pas être visibles dans le code
- La configuration de la CICD va se ralonger

Rendu - Déployer un Nuxt

Partie 5

Expliquer moi tout ce que vous avez fait via un schéma

- Utiliser draw.io
- Schématiser l'infra AWS
- Schématiser la CICD
- Schématiser tout ce qui est possible

Next

Projet

Next

Le contexte

- Je souhaite faire développer une application web de gestion de projet
- Vous avez été missionnés pour concevoir et développer cette application
- L'objectif est de créer une première version fonctionnelle de la plateforme
- L'application doit permettre à des équipes de collaborer sur des projets
- Le projet devra être développé avec une architecture cloud serverless

User Stories (Users)

- En tant qu'utilisateur, je peux créer un compte avec un email unique afin d'accéder à la plateforme.
- En tant qu'utilisateur, je peux me connecter à mon compte afin d'accéder à mes équipes et mes projets.
- En tant qu'utilisateur, je peux consulter mon profil afin de voir mes informations.
- En tant qu'utilisateur, je peux modifier mon profil afin de mettre à jour mes informations.

User Stories (Teams)

- En tant qu'utilisateur, je peux créer une équipe afin de collaborer avec d'autres utilisateurs.
- En tant qu'utilisateur, je peux consulter les équipes auxquelles j'appartiens afin d'accéder à leurs projets.
- En tant qu'utilisateur, je peux consulter les membres d'une équipe afin de savoir qui y participe.

User Stories (Invitations)

- En tant que membre d'une équipe, je peux inviter un utilisateur à rejoindre l'équipe via son email afin de collaborer avec lui.
- En tant qu'utilisateur, je peux consulter mes invitations à rejoindre des équipes.
- En tant qu'utilisateur, je peux accepter une invitation afin de rejoindre l' équipe.
- En tant qu'utilisateur, je peux refuser une invitation afin de ne pas rejoindre l'équipe.

User Stories (Projects)

- En tant que membre d'une équipe, je peux créer un projet dans une équipe afin d'organiser le travail de cette équipe.
- En tant qu'utilisateur, je peux consulter les projets d'une équipe.
- En tant que membre d'une équipe, je peux consulter le détail d'un projet afin de voir ses tâches.
- En tant que membre d'une équipe, je peux modifier un projet afin de mettre à jour ses informations.
- En tant que membre d'une équipe, je peux supprimer un projet.

User Stories (Tasks)

- En tant que membre d'un projet, je peux créer une tâche dans un projet. (nom, description, statut)
- En tant que membre d'un projet, je peux consulter les tâches d'un projet.
- En tant que membre d'un projet, je peux modifier une tâche.
- En tant que membre d'un projet, je peux supprimer une tâche.
- En tant que membre d'un projet, je peux assigner une tâche à un membre de l'équipe.
- En tant que membre d'un projet, je peux changer le statut d'une tâche (todo / in progress / done).

User Stories (Assets)

- En tant que membre d’un projet, je peux uploader un fichier sur une tâche.
- En tant que membre d’un projet, je peux consulter les fichiers liés à une tâche.
- En tant que membre d’un projet, je peux supprimer un fichier lié à une tâche.

Espace Admin

- L'application doit contenir un espace administrateur dédié
- Cet espace doit être un frontend séparé
- Accessible uniquement aux utilisateurs administrateurs

Fonctionnalités :

- Se connecter
- Consulter un dashboard avec statistiques globales (Nombre total d'utilisateurs, d'équipes, de projets et de tâches)
- Consulter tous les utilisateurs de la plateforme

Contraintes :

- Les routes doivent être protégées
- Vérification que l'utilisateur possède le rôle admin

Authentification

- Toute l'application doit utiliser JWT
- Lors du login, l'API retourne un token JWT
- Le token est envoyé dans les requêtes protégées

- Header attendu : Authorization

- Le JWT doit contenir l'id du user et doit s'expirer à un moment

Utilisation :

- protéger les routes utilisateur
- protéger les routes admin
- identifier l'utilisateur connecté

Stack à utiliser

Cloud / Infrastructure :

- Amazon S3 — hébergement des frontends et stockage des assets
- Amazon CloudFront — distribution des frontends
- Amazon API Gateway — exposition de l'API
- AWS Lambda — exécution de l'API et des crons
- Amazon RDS avec PostgreSQL
- Amazon SES pour l'envoie des mails
- Amazon Cognito

CI/CD :

- GitLab
- GitLab CI/CD

Stack à utiliser

Environnements :
- STG (staging)
- PRD (production)

Frontend :
- Framework frontend libre (React, Vue, etc.)
- CSS via tailwind

Backend :
- Runtime Bun
- Framework HTTP Hono
- Authentification via JWT
- API REST centralisée

Projet - Challenges

Features supplementaires

- Envoyer un mail d'invitation à une team via le service SES (+ var dynamique)
- Créer un cron qui se lance toutes les heures pour faire un backup de la base de donnée dans un S3
- Voir la liste des backups dans l'espace admin (date, heure, lien vers le fichier dans le S3)

Le front n'est pas très important

Contraintes techniques supplementaires

- Reprendre la même structure de répo que le cours dernier
- ORM interdit (donc script de migrations manuel)
- Tous les scripts de CICD doivent pouvoir être lancé en local
- Les lambdas hono doivent pouvoir se lancer en local sur un port du localhost
- Il faut faire remonter des logs dans AWS CloudWatch

- Les données des users doivent être détenu par le service Cognito
  - votre database ne possède que l'identifiant Cognito (sub)
  - l'api doit appeler Cognito pour connaître le nom et l'email d'un user
  - la connection front se faire via un package cognito (comme stripe)
  - il vous faut un service cognito avec des fonctions comme get_user_by_sub, get_user_by_email, update_user_by_sub ...

Structure technique principale

- code/**api**
- code/**crons**
- code/**domain** partagé entre api et crons pour la logique métier
- code/**emails** (template des emails)
- code/www-**assets**
- code/www-**user**
- code/www-**admin**

- Une **api gateway** avec une **lambda Hono + bun**
- Un service propre pour **aws_rds**
- Un service propre pour **aws_ses**
- Un service propre pour **aws_cognito**

Structure technique principale

- DockerFile et Docker-compose avec la **stack complète** pour développer

- Script deploy **front**
- Script deploy **assets**
- Script de **migrations** de base de données
- Script deploy **api**
- Script deploy **crons**

Pour les script **API et Crons**

1. Installer les packages bun
2. Build les lambdas hono (script dans package.json donné par Hono)
3. Envoyer sur aws

Présentation (vendredi)

Élève 1

1. Supprimer et re-build le dev container
2. Lancer l'api en local (+ me montrer que ça marche avec postman)
3. Lancer le script de migrations
4. Lancer un script de deploy api
5. M'expliquer Cognito

+ Questions

Présentation (vendredi)

Élève 2

1. Me montrer que le site user est en ligne
2. Me montrer la feature des invitations
3. Me montrer la feature des pièces jointes
4. Lancer le script deploy assets (+ avec un nouveau logo par exemple)
5. Lancer un script deploy front

+ Questions

Présentation (vendredi)

Élève 3

1. Me montrer que le site admin est en ligne
2. Me montrer que les backups marchent (en lancant le cron en local)
3. Lancer le script de deploy crons
4. Me montrer la CICD avec un push
5. Me montrer la gestions des environments
6. Me montrer les remontés de logs

+ Questions

Projet - RENDU

À déposer sur DVL

Par groupe de 3 maximum (1 dépôt par groupe est suffisant)

- Noms des membres du groupe
- Lien du repository public sur GitLab
- URLs CloudFront Front User **(stg et prd)**
- URLs CloudFront Front Admin **(stg et prd)**
- Screens aws pour tous les services utilisés

ATTENTION, SUPPRIMER TOUS VOS SERVICES AWS DANS UNE SEMAINE
(maintenez une liste avec tous vos services actifs qu'il faudra supprimer)

Guidage du projet (ordre conseillé)

1. Créer le repository Git
2. Mettre en place l'infrastructure cloud
3. Créer les scripts requis dans /infrastructure/scripts
4. Mettre en place les environnements
5. Mettre en place les migrations de base de données
6. Mettre en place la CI/CD

7. Deploy une version vide de l'infrastructure

8. Développer l'API
9. Développer le Cron
10. Développer le Front User
11. Développer le Front Admin

Structure de la base de données

Tables principales :

- users
- teams
- team_members
- invitations
- projects
- tasks
- assets
- backups

Relations :

- Un user peut appartenir à plusieurs teams
- Une team possède plusieurs members
- Une team possède plusieurs projects
- Un project appartient à une team
- Un project possède plusieurs tasks
- Une task peut être assignée à un user
- Une task peut contenir plusieurs assets
- Une team peut envoyer plusieurs invitations

Contraintes importantes :

- users.email doit être unique
- Une invitation est liée à un email
- Une task doit appartenir à un project
- Un project doit appartenir à une team

Liste des endpoints API

- **POST** /users
- **POST** /auth/login
- **GET** /me
- **PATCH** /me
- **POST** /teams
- **GET** /teams
- **GET** /teams/:teamId
- **GET** /teams/:teamId/members
- **POST** /teams/:teamId/invitations
- **GET** /invitations
- **POST** /invitations/:invitationId/accept
- **POST** /invitations/:invitationId/reject
- **POST** /teams/:teamId/projects
- **GET** /teams/:teamId/projects
- **GET** /projects/:projectId
- **PATCH** /projects/:projectId
- **DELETE** /projects/:projectId
- **POST** /projects/:projectId/tasks
- **GET** /projects/:projectId/tasks
- **GET** /tasks/:taskId
- **PATCH** /tasks/:taskId
- **DELETE** /tasks/:taskId
- **PATCH** /tasks/:taskId/assign
- **PATCH** /tasks/:taskId/status
- **POST** /tasks/:taskId/assets
- **GET** /tasks/:taskId/assets
- **DELETE** /assets/:assetId

- **GET** /admin/users - /stats - /backups